



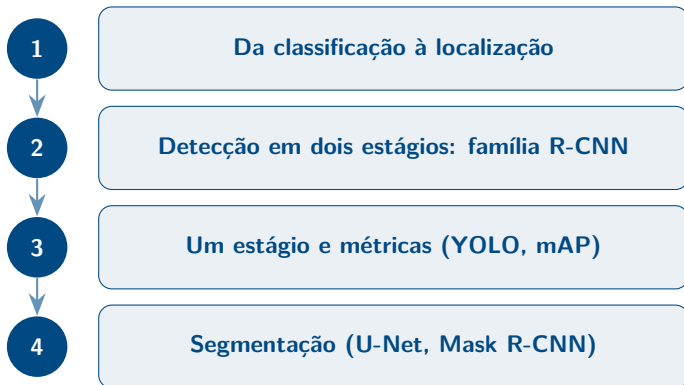
Localização, Detecção e Segmentação de Objetos

Aprendizagem de Máquina — CDIA

Prof. Dr. Rooney R. A. Coelho

Graduação em Ciência de Dados e Inteligência Artificial — CDIA

2026



Slides



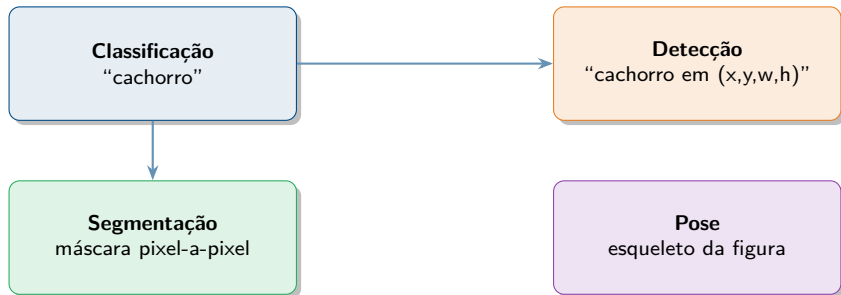
4 notebooks (executáveis com MPS)



Stevens et al., cap. 13

De onde paramos: classificação responde “o que”

A aula de CNNs ensinou a responder “o que tem na imagem?”. Hoje vamos além: “o que tem e onde está?” — e descer até o **pixel**.



A CNN não muda — ganha cabeças novas e perdas novas.

Objetivos

Ao final desta aula, você será capaz de:

- Representar caixas, calcular **IoU** e treinar um modelo que faz **classificação + localização**.
- Entender a família **R-CNN** (RPN, anchors, RoI Align, FPN).
- Distinguir detectores de **um estágio** (YOLO, SSD) dos de **dois estágios**, e interpretar **mAP**.
- Aplicar **segmentação semântica** (U-Net) e **de instâncias** (Mask R-CNN).
- Usar modelos prontos de **torchvision** e **ultralytics**.

Promessa

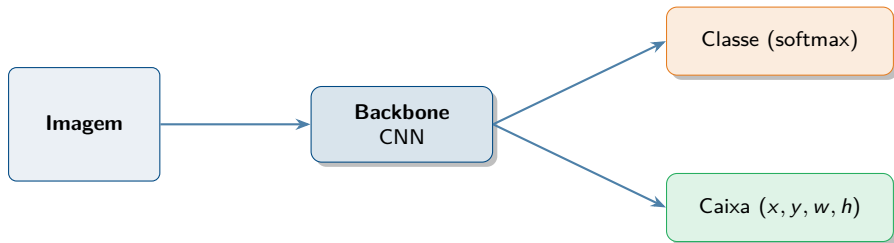
No fim, ao ver “YOLOv8” ou “mAP@0.5”, você sabe exatamente o que cada coisa significa e o que ajustar.



Da classificação à localização

De “o que” para “o que e onde”

A CNN já sabe responder **o que** aparece. Falta dizer **onde** — e isso é um pequeno acréscimo na cabeça da rede.



Ideia central

A CNN não muda — ela apenas **ganha um segundo ramo** que regride 4 números: a caixa do objeto.

Bounding box: 3 jeitos de escrever a mesma caixa

Uma caixa é **4 números**. Mas *quais* 4?

Formato	Significado	Quem usa
<code>xyxy</code>	(x_1, y_1, x_2, y_2) canto-canto, pixels	torchvision, COCO eval
<code>cxcywh</code>	centro (c_x, c_y) + largura/altura	YOLO (normalizado em $[0, 1]$)
<code>xywh</code>	canto sup. esq. + largura/altura	COCO (anotação JSON)

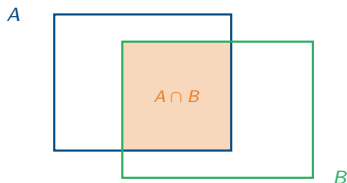
Cuidado!

Misturar formatos é o **bug nº 1** em detecção. Sempre confira: *xyxy ou cxcywh? pixels ou normalizado?*

IoU — Intersection over Union

Como medir se duas caixas “concordam”? Dividimos a área da **interseção** pela área da **união**:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} \in [0, 1]$$

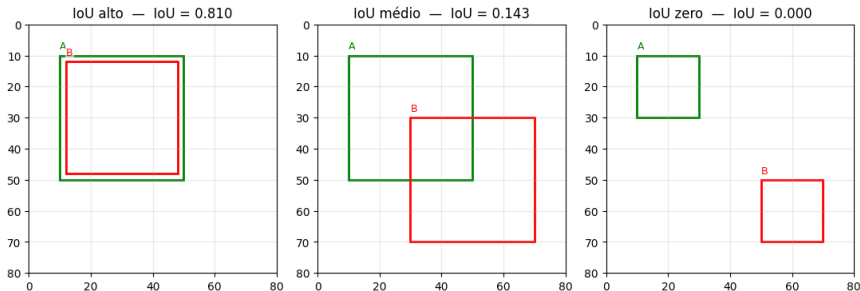


Convenção

Em detecção, costuma-se chamar de “acerto” quando $\text{IoU} \geq 0,5$ entre a caixa prevista e a verdadeira.

IoU em ação — três casos do notebook

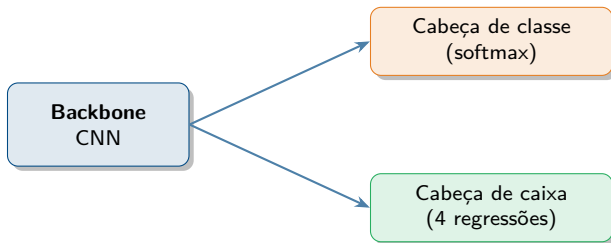
Os três pares de caixas abaixo foram desenhados pelo [notebook 01](#). Verde é a caixa A; vermelho é a caixa B.



Como ler cada caso

IoU $\approx 0,81$: caixas quase coincidem — seria um acerto. **IoU $\approx 0,14$:** tocam só uma faixa — contaria como erro de localização. **IoU = 0:** disjuntas. Convenção padrão em detecção: aceitar como acerto a partir de **IoU $\geq 0,5$** .

Classificação + Localização = *multi-tarefa*



$$L = L_{\text{classe}} + \lambda \cdot L_{\text{caixa}}$$

Quais perdas?

Classe: *CrossEntropy* (tarefa de classificação). **Caixa:** *Smooth L1* (Huber) — quadrática perto de zero, linear longe: robusta a *outliers*.

Em código: modelo + perda multi-tarefa

Um esqueleto enxuto em PyTorch:

```
class Localizador(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        base = models.resnet18(weights="DEFAULT")
        self.backbone = nn.Sequential(*list(base.children())[:-1])
        self.cabeca_classe = nn.Linear(512, num_classes)
        self.cabeca_caixa = nn.Linear(512, 4)

    def forward(self, x):
        f = self.backbone(x).flatten(1)
        return self.cabeca_classe(f), self.cabeca_caixa(f)

# Perda multi-tarefa (1 classe + 1 caixa por imagem)
LAMBDA = 1.0
logits, caixa = modelo(imgs)
perda = F.cross_entropy(logits, y_classe) \
    + LAMBDA * F.smooth_l1_loss(caixa, y_caixa)
```

E se houver N objetos na imagem?

O modelo anterior produz **tamanho fixo**: sempre 4 números de caixa e um vetor de classes.

Mas o mundo real tem **N variável**:

- uma foto com 1 pessoa;
- outra com 12 carros estacionados;
- outra ainda com 0 objetos da classe de interesse.

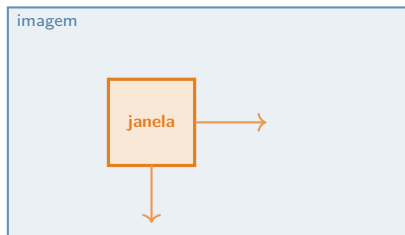
Não dá para a rede “cuspir $4N$ saídas”: N depende da imagem.

O problema é estrutural

Precisamos de uma ideia **qualitativamente diferente** — não basta repetir o classificador.

Ideia ingênua: janela deslizante

Por que não *varrer* a imagem com uma janela e classificar cada posição?



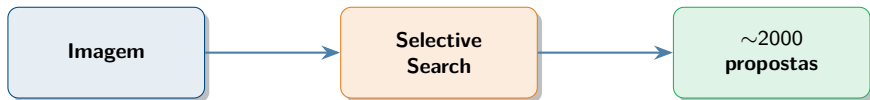
Problema: para cobrir objetos de qualquer tamanho e proporção, é preciso varrer **múltiplas escalas** e **razões de aspecto** $\Rightarrow$ *milhões* de janelas por imagem.

Conclusão

Inviável computacionalmente — e a maioria das janelas é fundo. Precisamos selecionar **poucas regiões promissoras**.

Region proposals: *Selective Search*

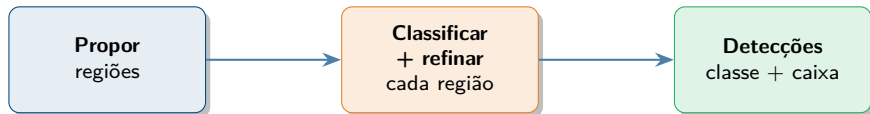
Em vez de testar todas as janelas, geramos cerca de **2000 regiões promissoras** por imagem. O *Selective Search* agrupa *superpixels* vizinhos com cor e textura parecidas.



O ganho

De *milhões* de janelas para *milhares* de candidatas — todas com chance razoável de conter algo.

O paradigma: propor $\rightarrow$ classificar $\rightarrow$ refinar

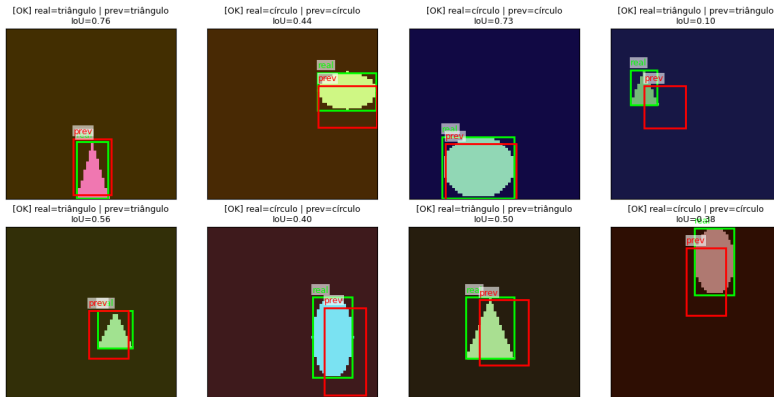


Esse é o esquema

Essa receita “propor $\rightarrow$ classificar” é exatamente o que faz a família **R-CNN** — nosso próximo bloco.

O localizador treinado — previsões reais

Modelo classe + caixa do slide anterior, treinado no [notebook 01](#). Caixa **real** vs **prevista**; IoU no título.



O que observar

Acurácia da classe $\approx 100\%$; IoU médio $\approx 0,66$. *Classificar* é fácil; *localizar* a borda exata é o que custa.

Resumo do bloco 1

- Uma **caixa** = 4 números (cuidado com o formato!).
- **IoU** mede sobreposição; $\geq 0,5$ é o limiar usual.
- Localização é **multi-tarefa**: CrossEntropy + Smooth L1.
- Janela deslizante é **inviável** para N objetos arbitrário.

Próximo bloco

R-CNN, Fast R-CNN, Faster R-CNN: como transformar “propor $\rightarrow$ classificar” em um detector que realmente funciona.

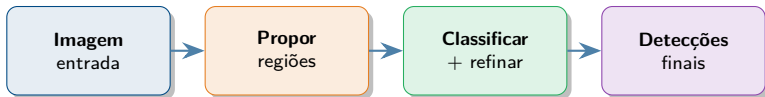


Detecção em dois estágios: a família R-CNN

Detecção em dois estágios: a ideia

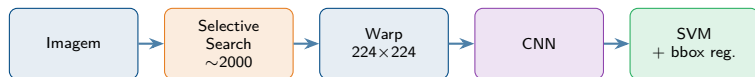
A receita é simples e intuitiva:

- 1 **Propor** regiões que *parecem* conter objetos.
- 2 **Classificar** cada região e **refinar** a caixa.



Vamos ver como a família R-CNN evoluiu essa ideia.

R-CNN (Girshick et al., 2014)



Três problemas práticos

- (a) lento — **2000 forwards** por imagem;
- (b) treino multi-etapa (CNN, SVMs, regressores);
- (c) Selective Search é fixo — *não aprende*.

Fast R-CNN (2015): 1 CNN + RoI Pooling



As ~ 2000 propostas agora são recortadas **no feature map**, não na imagem original. Treino **end-to-end**, muito mais rápido — e melhor acurácia.

O gargalo restante: Selective Search

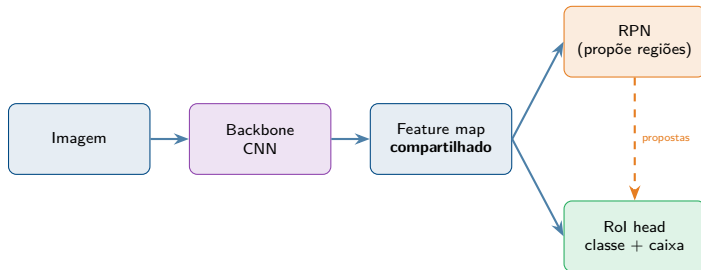
A CNN ficou rápida. Mas o **Selective Search** ainda roda na **CPU** e leva ~ 2 s por imagem — mais do que a própria CNN.

E pior: SS não aprende

Selective Search é um algoritmo *fixo* de visão clássica. Não se adapta ao dataset, não usa GPU e não melhora com mais dados.

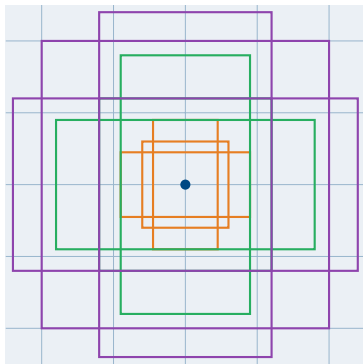
Próximo passo natural: fazer da **proposta de regiões** também uma **rede neural**.

Faster R-CNN (2015): a RPN



Tudo virou rede: **treina junto, roda na GPU**. A proposta de regiões deixou de ser um algoritmo fixo.

Anchor boxes



9 âncoras
3 escalas
×
3 razões

Como a rede usa as âncoras

A RPN não inventa caixas do zero: ela prevê **ajustes (deltas)** sobre cada âncora pré-definida.

RPN: previsões e treino

Para cada âncora, a RPN prevê:

- **Objectness** — probabilidade de conter *algum* objeto (vs. fundo). *Classificação binária*.
- **4 deltas** (t_x, t_y, t_w, t_h) — ajustes a aplicar sobre a âncora.

Atribuição de rótulos:

- IoU *alto* com alguma caixa GT $\Rightarrow$ **positivo**.
- IoU *baixo* com todas as GT $\Rightarrow$ **negativo**.

$$\mathcal{L}_{\text{RPN}} = \mathcal{L}_{\text{cls}}(\text{obj/fundo}) + \lambda \cdot \mathcal{L}_{\text{reg}}(\text{deltas, só nas positivas})$$

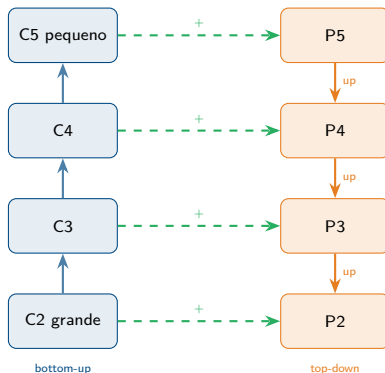
Rol Pooling (Fast R-CNN) **arredonda** coordenadas para células inteiras do feature map $\Rightarrow$ **desalinhamento** de até alguns pixels.

Rol Align usa **interpolação bilinear** para amostrar nos pontos exatos $\Rightarrow$ alinhamento preciso entre Rol e features.

Por que isso importa

Essencial para **objetos pequenos** e, sobretudo, para **máscaras pixel-a-pixel** — volta no Mask R-CNN (bloco 4).

FPN — Feature Pyramid Network



Pirâmide com **semântica forte em todas as escalas** — detecta objetos *pequenos e grandes* no mesmo modelo.

NMS — limpando duplicatas

A rede gera **várias caixas** para o mesmo objeto (âncoras vizinhas convergem para a mesma região).

Non-Maximum Suppression (NMS):

- Ordena as caixas por **score** (decrecente).
- Mantém a de maior score.
- Descarta as que têm **IoU alto** com ela.
- Repete até esgotar a lista.

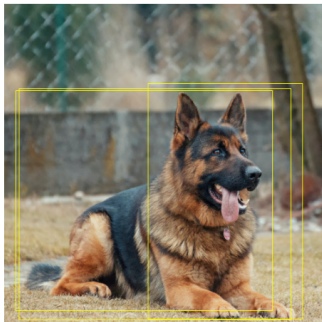
Onde NMS aparece

Toda família R-CNN e todo YOLO usam NMS no final. Veremos o **algoritmo completo** e suas variantes no **bloco 3**.

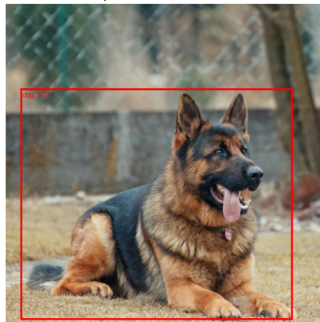
NMS na prática — 3 caixas viram 1

Forçamos o Faster R-CNN a mostrar duplicatas: limiar de score 0.05 + `torchvision.ops.nms(boxes, scores, iou_threshold=0.3)`.

Antes do NMS — 3 caixas



Depois do NMS — 1 caixas



Leitura

Esquerda: 3 caixas sobrepostas para o mesmo cachorro. **Direita:** NMS manteve só a de maior score. Na inferência normal isso acontece dentro do modelo.

Faster R-CNN em código ()

```
import torch
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

# 1) Modelo pre-treinado em COCO
modelo = fasterrcnn_resnet50_fpn(weights="DEFAULT")
modelo.eval()

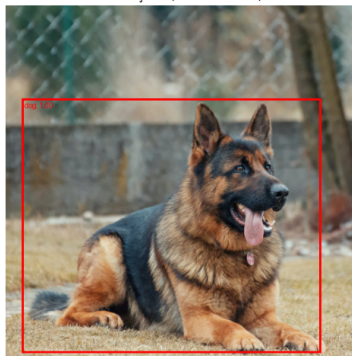
# 2) Inferencia: lista de imagens -> lista de dicts
with torch.no_grad():
    saidas = modelo([img]) # img: tensor [3,H,W]
    boxes = saidas[0]["boxes"] # xyxy em pixels
    labels = saidas[0]["labels"]
    scores = saidas[0]["scores"]

# 3) Fine-tuning: trocar a cabeça de classificacao
in_feats = modelo.roi_heads.box_predictor.cls_score.in_features
modelo.roi_heads.box_predictor = FastRCNNPredictor(in_feats, num_classes=N+1)
# +1 porque o indice 0 e o fundo
```

Faster R-CNN pré-treinado em ação

Em 5 linhas de código e *sem treinar nada*, o modelo pré-treinado em COCO encontra o cachorro com **100 % de confiança**.

Detecções (score ≥ 0.5)



Ponto de partida de qualquer projeto

As 80 classes do COCO cobrem a maior parte de “foto natural” (pessoa, animais, veículos).
Fine-tuning entra só quando a sua classe não está em COCO.

Resumo do bloco 2

- **R-CNN** → **Fast** → **Faster**: cada versão corrige um gargalo (CNN repetida, treino multi-etapa, Selective Search).
- **RPN + anchors**: a proposta de regiões virou uma rede, prevendo ajustes sobre âncoras pré-definidas.
- **RoI Align**: amostragem precisa, sem arredondamento.
- **FPN**: semântica em todas as escalas ⇒ objetos pequenos e grandes.

Próximo bloco

Detectores de **um estágio** (YOLO) — sem proposta separada — e as **métricas** (NMS detalhado, IoU, mAP).



Detecção em um estágio e métricas

Um estágio: pular as propostas

A família **R-CNN é precisa**, mas trabalha em **dois passos**: primeiro propor regiões, depois classificar cada uma. Isso custa tempo — mesmo o Faster R-CNN dificilmente passa de **~10 FPS** em hardware comum.

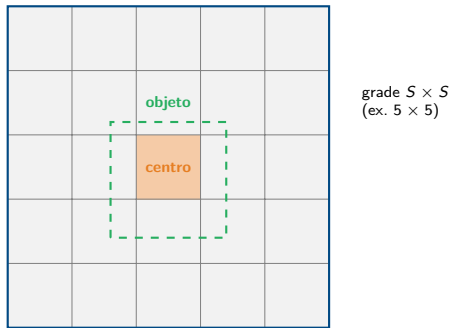
Para **vídeo, robótica e tempo real** queremos uma única passada da CNN, sem propostas separadas.

A grande ideia (YOLO, 2016)

Tratar a detecção como **um problema de regressão**: em *uma* passada, a rede prevê *todas* as caixas e classes.

YOLO: a grade e a previsão

A imagem é dividida em uma grade $S \times S$. A célula que contém o **centro** do objeto é responsável por prevê-lo.



Cada célula prevê B caixas (x, y, w, h, conf) e C probabilidades de classe. Saída total: tensor $S \times S \times (B \cdot 5 + C)$.

YOLO v1: loss e limitações

A **loss** é uma única soma de erros quadráticos com três componentes:

$$\mathcal{L} = \lambda_{\text{coord}} \mathcal{L}_{\text{coord}} + \mathcal{L}_{\text{conf}} + \lambda_{\text{noobj}} \mathcal{L}_{\text{noobj}} + \mathcal{L}_{\text{classe}}$$

Detalhes-chave:

- Usa $\sqrt{w}$ e $\sqrt{h}$: erro relativo pesa mais em caixas pequenas.
- $\lambda_{\text{coord}} \approx 5$ e $\lambda_{\text{noobj}} \approx 0,5$ equilibram localização vs. fundo.

Limitação histórica

Apenas B caixas por célula $\Rightarrow$ ruim com **objetos pequenos** e **aglomerados** (rebanho, multidão).

SSD e *anchors* em um estágio

O **SSD** (2016) detecta em **vários mapas de features**: camadas rasas pegam objetos pequenos, camadas profundas pegam grandes.

A partir do YOLOv2 e do SSD, os detectores adotam **anchors**: caixas de referência em escalas e proporções fixas, ancoradas nas células da grade.

Por que *anchors* estabilizam o treino

A rede não prevê coordenadas cruas: prevê **ajustes** ($\Delta x, \Delta y, \Delta w, \Delta h$) sobre âncoras de referência. Mesma ideia da RPN do Faster R-CNN.

Evolução do YOLO + anatomia moderna



Toda versão moderna segue a mesma anatomia: **backbone** (CNN) → **neck** (FPN) → **head** (caixas + classes).

Vocabulário familiar

É exatamente o mesmo **backbone/head** da aula de CNN — só muda o que a cabeça prevê.

Um estágio vs dois estágios: *trade-off*

	Dois estágios (Faster R-CNN)	Um estágio (YOLO, SSD)
Precisão (mAP)	mais alta historicamente	competitiva hoje
Velocidade	~5–10 FPS	~30–150 FPS
Pipeline	RPN + cabeça	uma passada
Uso típico	batch, alta precisão	vídeo, robótica, edge

Hoje a diferença em mAP encolheu muito — YOLOv8/v11 e RT-DETR competem de frente com detectores em dois estágios.

Focal Loss + RetinaNet

Em um estágio, **a maioria das âncoras é fundo**. O gradiente desses exemplos fáceis *domina* e o modelo nunca aprende bem os objetos.

A **Focal Loss** reduz o peso de exemplos já bem classificados:

$$\text{FL}(p) = -(1 - p)^\gamma \log(p), \quad \gamma \approx 2$$

Quando p é alto (exemplo fácil), $(1 - p)^\gamma$ é pequeno.

RetinaNet (2017)

Um estágio + **FPN** + **Focal Loss**: o primeiro detector de uma passada a *bater* a precisão dos de dois estágios.

A rede prevê **centenas a milhares de caixas** por imagem. A **Non-Maximum Suppression** fica com a melhor por objeto.

- 1 **Ordenar** as caixas por *score* de confiança (maior $\rightarrow$ menor).
- 2 **Pegar** a caixa de maior *score* e marcá-la como mantida.
- 3 **Descartar** todas as caixas com **IoU** $>$ **limiar** (ex. 0,5) em relação à mantida.
- 4 **Repetir** 2–3 nas caixas restantes até esvaziar a lista.

Resultado: no máximo uma caixa por objeto, com o maior *score*.

NMS em código

```
import numpy as np

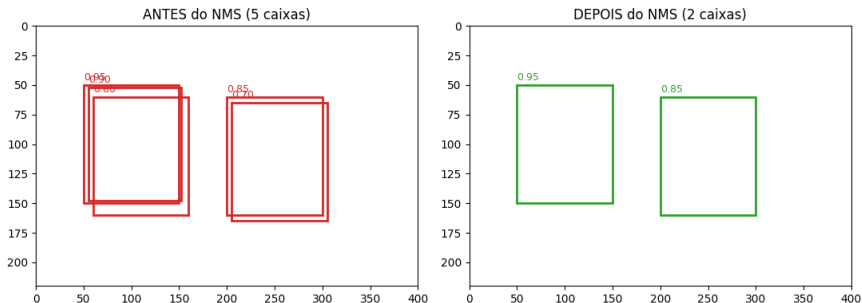
def nms(caixas, scores, limiar_iou=0.5):
    # caixas: (N, 4) em xyxy ; scores: (N,)
    idx = np.argsort(scores)[::-1] # maior score primeiro
    mantidas = []
    while len(idx) > 0:
        i = idx[0]
        mantidas.append(i)
        ious = iou_vetorizada(caixas[i], caixas[idx[1:]])
        idx = idx[1:][ious < limiar_iou] # descarta sobrepostas
    return mantidas
```

Na prática

`torchvision.ops.nms(caixas, scores, iou_threshold)` faz exatamente isso, otimizado em C++/CUDA.

NMS passo a passo — 5 caixas viram 2

Conjunto-brinquedo do [notebook 03](#): 5 propostas com scores diferentes cobrindo *dois* objetos.



O algoritmo, executado nesta imagem

1) Pega a melhor (**0,95**) → vira detecção final. 2) Descarta tudo com $\text{IoU} \geq 0,5$ contra ela (sumiram a 0,90 e a 0,80). 3) Repete com o que sobrou → entra a 0,85. 4) Para. *O nosso NMS “do zero” devolve os mesmos índices que `torchvision.ops.nms`.*

DETR — detecção como predição de conjunto

O **DETR** (Facebook, 2020) reformula tudo com um **Transformer**:

- CNN extrai *features* → Transformer encoder/decoder.
- Prevê um **conjunto fixo** de N caixas (ex. $N=100$).
- Treino com **bipartite matching**: cada GT é casada com *uma* predição (Hungarian algorithm).
- **Sem anchors. Sem NMS.**

Por que importa

Pipeline radicalmente mais simples e ponte natural para os **Transformers visuais** — assunto de aulas futuras.

Avaliação: TP, FP, FN

Uma detecção é considerada **correta** (TP) se atende a **duas** condições:

- (a) a **classe predita** bate com a do *ground truth*;
- (b) o **IoU** com uma GT **ainda não emparelhada** é $\geq$ um limiar (ex. 0,5).

- **TP** — detecção válida, classe certa, IoU suficiente.
- **FP** — detecção sem GT correspondente (ou duplicada).
- **FN** — objeto real *não* detectado pela rede.

Não há “TN” significativo em detecção — o fundo é praticamente infinito.

Com TP, FP e FN definimos:

$$\text{Precisão} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Variando o **limiar de score**, traçamos a **curva PR** (Precisão $\times$ Recall) de cada classe.

- **AP** (*Average Precision*) = área sob a curva PR de uma classe.
- **mAP** = média das AP sobre todas as classes.

Duas convenções no mundo real

mAP@0.5 (PASCAL VOC): IoU fixo em 0,5. **mAP@[.5:.95]** (COCO): média do mAP em IoUs de 0,5 a 0,95 (passo 0,05) — bem mais rigoroso.

YOLO em código ()

```
from ultralytics import YOLO

# pre-treinado em COCO (80 classes)
modelo = YOLO("yolov8n.pt")

# inferencia -- NMS ja eh aplicada internamente
resultados = modelo.predict("img.jpg", conf=0.25)

# treino / fine-tuning
modelo.train(data="data.yaml", epochs=50, imgsz=640)
```

Formato dos dados

Pasta `images/` + pasta `labels/` (um `.txt` por imagem com `classe xc yc w h normalizados`)
+ um `data.yaml` com os caminhos e a lista de classes.

YOLOv8 numa passada — 6 objetos detectados

bus.jpg processada por uma **YOLOv8 nano** (~6 MB) em MPS:

YOLOv8n — deteccoes



O que o modelo “viu”

1 **ônibus** (0,87) + 4 **pessoas** (0,87/0,85/0,83/0,26) + 1 **placa de pare**. Uma passada de `forward`; NMS dentro do `predict()`.

Resumo do bloco 3

- **Um estágio** (YOLO, SSD): uma passada $\Rightarrow$ tempo real.
- **Focal Loss / RetinaNet**: resolve o desbalanço fundo/objeto.
- **NMS**: filtra caixas duplicadas por IoU.
- **mAP**: padrão de avaliação — PASCAL (0,5) e COCO (.5:.95).

Próximo bloco

Da caixa para o **pixel**: segmentação semântica (U-Net) e segmentação de instâncias (Mask R-CNN).



Segmentação semântica e de instâncias

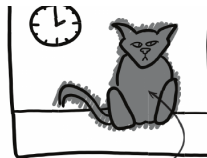
Da caixa à forma exata

A **detecção** responde *onde* com um retângulo. A **segmentação** responde com a *forma exata* do objeto — pixel a pixel.

“Cat: yes” (classificação) vs. “Cat: *here*” (segmentação).



CAT: YES



CAT: HERE

Granularidade

Caixa → máscara: a resposta vira uma *imagem*.

Três tipos de segmentação

“Segmentação” é um termo guarda-chuva. Há **três sabores** distintos — e perdas/saídas diferentes para cada um.

Semântica

classe por pixel
não separa objetos

Instâncias

máscara por objeto
separa cada um

Panóptica

semântica + instâncias
visão unificada

Foco de hoje

Semântica (U-Net) e **instâncias** (Mask R-CNN).

Segmentação semântica: classe por pixel

- **Entrada:** imagem $H \times W \times 3$.
- **Saída:** mapa $H \times W$ com um rótulo por pixel.
- **Loss:** *CrossEntropy* aplicada *pixel a pixel*.
- Treinamento idêntico ao de classificação — só repetido $H \cdot W$ vezes.

$$L = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W \text{CE}(\hat{y}_{ij}, y_{ij})$$

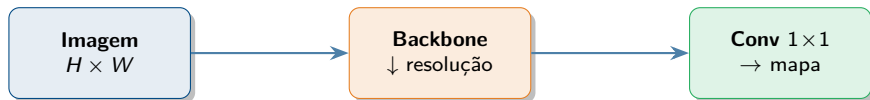
Em uma frase

É *literalmente* a classificação aplicada a cada um dos $H \cdot W$ pixels da imagem.

Downsampling + FCN

O backbone reduz H, W (pooling/stride) para ganhar **semântica** em cada ativação. Mas a saída precisa voltar a $H \times W$.

A **FCN** (Long et al., 2015) trocou as camadas *fully connected* finais por convoluções $1 \times 1 \Rightarrow$ produz um **mapa**, não um vetor.



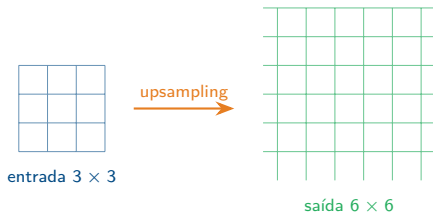
Vantagem da FCN

Sem camadas FC, a rede aceita **imagens de qualquer tamanho**.

Upsampling: convolução transposta

Para voltar a $H \times W$ temos duas opções:

- **Interpolação** (*bilinear, nearest*): rápida, sem parâmetros.
- **Conv transposta** (`ConvTranspose2d`): aprendível, expande o mapa.



Atenção

Cuidado com artefato de **tabuleiro de xadrez** se kernel e stride não encaixarem (use stride que divide o kernel).

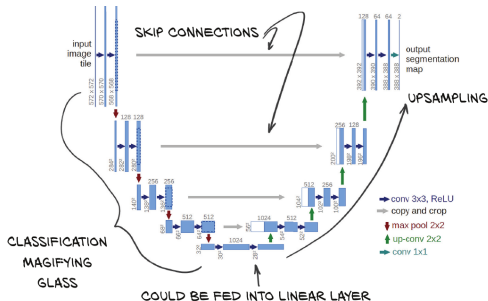
U-Net (Ronneberger et al., 2015)

Encoder à esquerda (desce a resolução, sobe canais). **Decoder** à direita (sobe a resolução).

Skip connections atravessam o “U” — trazem detalhe espacial fino do encoder direto para o decoder.

Por que funciona

Skips preservam contornos. Ideal para **poucos dados rotulados** (origem médica).



Mini U-Net em código (esqueleto)

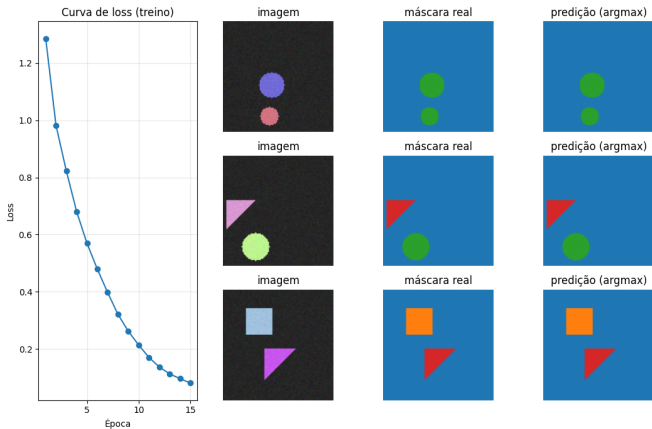
```
class DuplaConv(nn.Module): # Conv -> BN -> ReLU (2x)
    def __init__(self, ent, sai):
        super().__init__()
        self.bloco = nn.Sequential(
            nn.Conv2d(ent, sai, 3, padding=1), nn.BatchNorm2d(sai), nn.ReLU(True),
            nn.Conv2d(sai, sai, 3, padding=1), nn.BatchNorm2d(sai), nn.ReLU(True))
    def forward(self, x): return self.bloco(x)

class MiniUNet(nn.Module):
    def __init__(self, n_cls):
        super().__init__()
        self.d1 = DuplaConv(3, 64); self.d2 = DuplaConv(64, 128)
        self.pool = nn.MaxPool2d(2)
        self.up = nn.ConvTranspose2d(128, 64, 2, stride=2)
        self.u1 = DuplaConv(128, 64) # 64 (up) + 64 (skip)
        self.saida = nn.Conv2d(64, n_cls, 1)

    def forward(self, x):
        x1 = self.d1(x); x2 = self.d2(self.pool(x1))
        u = self.up(x2)
        u = torch.cat([u, x1], dim=1) # skip connection
        return self.saida(self.u1(u))
```

Mini U-Net treinada — loss e previsões

Mini U-Net do slide anterior, treinada no [notebook 04](#) (15 épocas em MPS).



Como ler

Esquerda: loss 1,0 $\rightarrow$ 0,01 (CrossEntropy por pixel). **Direita:** predição ([argmax](#)) bate com a real pixel a pixel. *Tão bom é típico de dados sintéticos; em fotos reais mIoU $\sim$ 0,5–0,9.*

Métricas de segmentação

Métrica	Fórmula	Observação
Pixel accuracy	$\# \text{corretos} / \# \text{pixels}$	engana com classes desbalanceadas
IoU / Jaccard	$ A \cap B / A \cup B $	por classe; mIoU = média (padrão)
Dice	$2 A \cap B / (A + B)$	pesa mais a interseção

Dica prática

Dice loss = $1 - \text{Dice}$ funciona bem para *classes raras* — padrão em segmentação *médica*.

Segmentação semântica pronta (torchvision)

```
from torchvision.models.segmentation import deeplabv3_resnet50

modelo = deeplabv3_resnet50(weights="DEFAULT").eval()

with torch.no_grad():
    saida = modelo(imagem) # imagem: [N, 3, H, W]

logits = saida["out"] # [N, C, H, W] (C = 21 no VOC)
mapa = logits.argmax(dim=1) # [N, H, W] classe por pixel
```

Observação

A saída é um `dict`: pegue `saida["out"]` e aplique `argmax(1)` no eixo das classes para obter o **mapa de rótulos**.

DeepLabV3 pré-treinado — semântica

`deeplabv3_resnet50` com pesos COCO/VOC numa foto baixada na hora pelo [notebook 04](#).

imagem



DeepLabV3 (argmax)

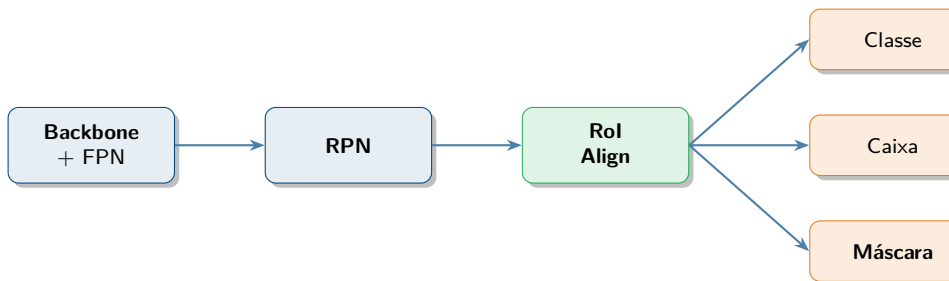


Leitura

Sem treino nosso o modelo separou **ônibus** (vermelho), **pessoas** (cinza) e **fundo** (azul). Saída: (1, 21, 256, 256) — 21 classes VOC; mapa visual: `saida["out"].argmax(1)`.

Segmentação de instâncias: Mask R-CNN

Distinguir objetos da **mesma classe** = detecção + máscara *por objeto*. Extensão direta do Faster R-CNN.



Cabeça de máscara

Uma **pequena FCN** por RoI prediz uma máscara binária. **RoI Align** (sem arredondamento) é essencial para alinhar pixels.

Mask R-CNN em código

```
from torchvision.models.detection import maskrcnn_resnet50_fpn

modelo = maskrcnn_resnet50_fpn(weights="DEFAULT").eval()

with torch.no_grad():
    saidas = modelo(imagens) # lista de dicts, 1 por imagem

s = saidas[0]
caixas = s["boxes"] # [N, 4] xyxy
rotulos = s["labels"] # [N]
scores = s["scores"] # [N]
mascaras = s["masks"] # [N, 1, H, W] valores em [0, 1]
binarias = mascaras > 0.5 # limiar -> mascara binaria
```

Saída

Mesmo dict do Faster R-CNN + chave extra "masks".

Mask R-CNN — 6 instâncias separadas

Mesma foto, agora pelo `maskrcnn_resnet50_fpn`:

imagem



Mask R-CNN — 6 instâncias



Semântica $\neq$ instâncias

O DeepLabV3 dizia “*esses pixels são pessoa*”. O Mask R-CNN diz “*esta máscara é a pessoa 1; aquela, a pessoa 2*”. **6 instâncias** ($\geq 0,5$), cada uma com sua máscara binária da cabeça de máscara aplicada dentro de cada Rol Align.

- **Semântica** (classe por pixel) vs. **instâncias** (máscara por objeto).
- **FCN** + **conv transposta**: voltar a $H \times W$.
- **U-Net** + *skip connections*: detalhe espacial fino.
- **Mask R-CNN**: Faster R-CNN + cabeça de máscara.

Para onde o campo está indo

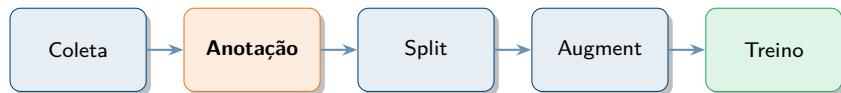
SAM (*Segment Anything*), detecção *open-vocabulary* (texto $\rightarrow$ caixa/máscara) e a ponte para **Transformers visuais** (DETR, Mask2Former).



Dados, anotação e ferramentas

Os dados importam tanto quanto o modelo

Você acabou de ver as quatro famílias de modelo. Mas **nenhum deles aprende nada** sem anotação. Em projeto real, $\sim 80\%$ do tempo é gasto em dados.



O que vem agora

Como organizar arquivos, **quais formatos** de anotação existem (COCO, VOC, YOLO), **quais programas** ajudam a anotar, e **quais boas práticas** evitam dor de cabeça.

Três formatos dominam: COCO, Pascal VOC, YOLO

Cada modelo / biblioteca espera um formato específico. Os três principais:

COCO

1 arquivo JSON
com tudo dentro

torchvision, pycocotools

Pascal VOC

1 arquivo XML
por imagem

formato clássico/legado

YOLO

1 TXT por imagem
+ [data.yaml](#)

ultralytics

Boa notícia

Toda ferramenta moderna (Roboflow, CVAT, Label Studio) exporta para os três. Você anota uma vez e converte conforme o modelo escolhido.

Formato COCO — JSON único

Um único arquivo JSON descreve tudo: imagens, anotações, classes.

```
{
  "images": [
    { "id": 1, "file_name": "img001.jpg", "width": 640, "height": 480 }
  ],
  "annotations": [
    { "id": 1, "image_id": 1, "category_id": 2,
      "bbox": [100, 50, 200, 150], "iscrowd": 0, "area": 30000 }
  ],
  "categories": [
    { "id": 1, "name": "pessoa" },
    { "id": 2, "name": "cachorro" }
  ]
}
```

Pontos importantes

bbox no formato `[x, y, w, h]` com canto superior esquerdo em *pixels*. `category_id` costuma começar em 1 (zero é fundo). É o formato esperado pelo `torchvision.models.detection`.

Formato Pascal VOC — XML por imagem

Um arquivo `.xml` por imagem dentro da pasta `Annotations/`.

```
<annotation>
  <filename>img001.jpg</filename>
  <size>
    <width>640</width> <height>480</height> <depth>3</depth>
  </size>
  <object>
    <name>cachorro</name>
    <bndbox>
      <xmin>100</xmin> <ymin>50</ymin>
      <xmax>300</xmax> <ymax>200</ymax>
    </bndbox>
  </object>
</annotation>
```

Pontos importantes

bndbox no formato `xyxy` (canto superior esquerdo + canto inferior direito) em *pixels*. Cada objeto vira um bloco `<object>`. Classe é um *nome*, não um índice. Formato do PASCAL VOC original (2007–2012), ainda comum em datasets clássicos.

Formato YOLO — TXT por imagem + data.yaml

Um `.txt` por imagem (1 linha por objeto) + um `data.yaml` descritor do dataset.

labels/img001.txt

```
1 0.453 0.260 0.312 0.312
0 0.812 0.611 0.180 0.225
```

classe x_c y_c w h

data.yaml

```
path: ./dataset
train: images/train
val: images/val
nc: 2
names: ['pessoa', 'cachorro']
```

Cuidado!

Coordenadas **normalizadas** em $[0, 1]$ (não pixels), formato **cxcywh** (centro + tamanho), classe é **índice de 0 a $N-1$** . Misturar com `torchvision` (xyxy em pixels) é o bug nº 1.

Comparativo dos três formatos

	COCO	Pascal VOC	YOLO
Arquivos	1 JSON único	1 XML por imagem	1 TXT por imagem + YAML
Coordenadas	xywh (pixels)	xyxy (pixels)	cxcywh (norm.)
Classe	id numérico	nome (texto)	índice 0..N-1
Verbosidade	média	alta	mínima
Ecosistema	torchvision , pycocotools	legado (Faster R-CNN, VOC)	ultralytics

Qual escolher

Modelo [torchvision](#) $\Rightarrow$ **COCO**. Modelo [ultralytics](#) $\Rightarrow$ **YOLO**. As ferramentas de anotação convertem entre os três automaticamente na exportação.

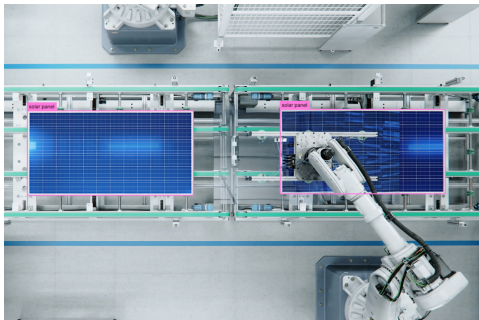
Ferramentas de anotação — panorama

Ferramenta	Aberta?	Hospedagem	Melhor para
Roboflow	✗ comercial (freemium)	na nuvem (web)	projetos rápidos, pipeline pronto
CVAT	✓ sim (MIT)	nuvem ou local (Docker)	equipes, datasets grandes, vídeo
Label Studio	✓ sim (Apache 2)	nuvem ou local	multimídia (texto, áudio, vídeo, imagem)
LabelImg	✓ sim (legado)	app local (Python)	casos simples, ensino

Todas elas

Exportam para **COCO, Pascal VOC e YOLO**. Suportam **caixas, polígonos e máscaras**. Diferem em UX, recursos de equipe, auto-anotação e modelo de hospedagem.

Roboflow — web-first, pipeline pronto



Exemplo: detecção de painéis solares numa linha de produção (*roboflow.com*).

Pontos fortes:

- Tudo na web — zero instalação.
- Versionamento de dataset embutido.
- Augmentation com 1 clique.
- Treina YOLO e exporta para deploy.
- Free tier generoso.

Limitações: fechado/comercial; planos pagos para volume.

CVAT — open source, robusto para equipes

CVAT

All-in-One Data Labeling
Suite for Teams Building
Real-World Vision AI

Get Started ↗

Quality control for Task #2147535

General

* Target metric

Accuracy

* Target metric threshold

70

Job Validation

* Max validations

3

Automatic annotation

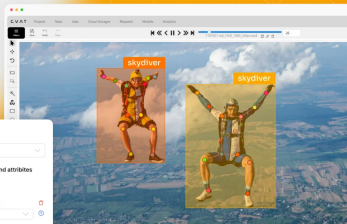
Model: SAM 3

Setup mapping between labels and attributes

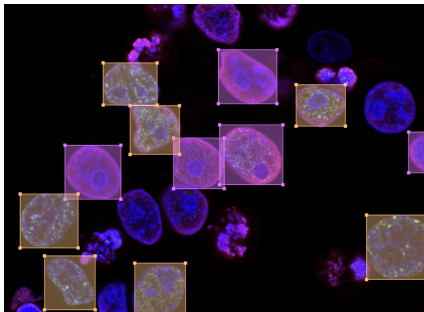
Model Spec → CVAT Spec

person

person



Pontos fortes: open source MIT; rode local com Docker ou use a versão gerenciada cvat.ai; excelente para **vídeo** (interpolação entre frames); integra modelos de **auto-anotação** (SAM, YOLO); usado por OpenCV, Intel e por muitos laboratórios acadêmicos.



Anotando uma imagem de microscopia (*labelstud.io*).

Pontos fortes:

- Open source (Apache 2), self-hosted ou cloud.
- Vai **muito além de imagem** : texto, áudio, vídeo, séries temporais.
- Templates prontos por tipo de tarefa.
- Plugins de ML para pré-anotação.

Bom para: laboratórios mistos (visão + NLP + áudio).

Workflow de anotação — boas práticas

Pipeline recomendado:

- 1 Definir as **classes** antes (lista fechada, sem “outro”).
- 2 Coletar 100–500 imagens diversas para começar.
- 3 Anotar com *guia visual* (exemplos do que é e do que não é).
- 4 Dividir **70 / 15 / 15** (treino / val / teste).
- 5 Treinar um *baseline* e **revisar os erros** — voltar a anotar onde o modelo erra mais (*active learning*).

Armadilhas comuns

Caixas inconsistentes entre anotadores (definir critério de borda); classes desbalanceadas (anotar mais da rara); “classe fundo” esquecida no índice 0 do [torchvision](#); *augmentation* que move a imagem mas não a caixa/máscara (use [albugmentations](#)).

Recapitulando

1

Localização
caixa, IoU, multi-tarefa

2

Dois estágios
R-CNN → Faster R-CNN, FPN

3

Um estágio + métricas
YOLO, NMS, mAP

4

Segmentação
U-Net, Mask R-CNN, Dice

Mensagem final

O **backbone CNN continua o mesmo**. O que muda é a *cabeça* e a *loss*.

Qual modelo usar na prática

Quando	Use	Por quê
Precisão máxima	Faster R-CNN + FPN	dois estágios refina melhor
Tempo real, vídeo	YOLO (v8/v11)	uma passada, muito rápido
Forma exata	Mask R-CNN	detecção + máscara
Classe por pixel	U-Net / DeepLabV3	segmentação semântica

Regra de bolso

Comece sempre por um modelo **pré-treinado em COCO** e faça *fine-tuning*. Treinar do zero raramente vale a pena.

- **Formato de caixa errado.** `torchvision`: xyxy em pixels. YOLO: cxcywh normalizado. Misturar = bug nº 1.
- **Esquecer o NMS** $\Rightarrow$ caixas duplicadas por objeto.
- **Augmentation que não move a caixa** ao girar/flipar $\Rightarrow$ rótulos errados (use `albumentations`).
- **num_classes sem o fundo** no `torchvision`: o índice 0 é reservado.
- **Comparar mAP entre datasets diferentes** — só é comparável no *mesmo* conjunto e *mesmo* critério de IoU.

Para continuar

- Stevens et al., *Deep Learning with PyTorch*, cap. 13–14.
- CS231n (Stanford), aula *Detection and Segmentation*.
- Papers na ordem: R-CNN → Fast → Faster R-CNN → YOLO → SSD → FPN → Mask R-CNN → RetinaNet → DETR.
- Docs: [torchvision.models.detection](#), [docs.ultralytics.com](#).
- Pacotes: [torchmetrics](#) (mAP pronto), [albumentations](#) (augmentation), [pycocotools](#).

Notebooks de hoje

[01_localizacao_e_iou](#) [02_deteccao_faster_rcnn](#) [03_yolo_e_metricas](#) [04_segmentacao](#)

Obrigado!

Dúvidas, sugestões e código no repositório do curso.

“A CNN diz *o que*; hoje você ensinou ela a dizer *onde*.”